

Relatório Técnico do Trabalho Prático 2 - Pareamento de Sequências de DNA

Isabela M. Andrade, Isadora R. Grandaux, Júlia A. da Rocha, Otávio de Oliveira,
Pedro Brassi Luccas.

¹Departamento de Ciências da Computação – Universidade Federal de Alfenas
Avenida Jovino Fernandes de Sales 2600 – CEP 37133840 - Alfenas – MG - Brasil

isabela.mageste@sou.unifal-mg.edu.br, isadora.grandaux@sou.unifal-mg.edu.br,
juliaalves.rocha@sou.unifal-mg.edu.br, otavio.oliveira1@sou.unifal-mg.edu.br,
pedro.luccas1@sou.unifal-mg.edu.br.

Resumo. Este trabalho aborda o problema de pareamento de sequências de DNA, no qual duas cadeias de bases são alinhadas com o objetivo de maximizar uma pontuação baseada em regras de correspondência. Foram implementados dois algoritmos distintos: um guloso e outro baseado em programação dinâmica. O algoritmo guloso realiza escolhas locais visando eficiência, enquanto a programação dinâmica garante a solução ótima. Foram realizados experimentos com diferentes tamanhos de entrada, analisando tempo de execução e consumo de memória. Os resultados mostram que o algoritmo guloso é significativamente mais rápido, porém apresenta baixa qualidade de solução, enquanto a programação dinâmica, embora mais custosa, obtém melhores pontuações, evidenciando o trade-off entre desempenho e otimalidade.

1. Introdução

O problema de alinhamento de sequências de DNA consiste em comparar duas sequências compostas pelas bases A, T, C e G, permitindo a inserção de lacunas para maximizar uma pontuação total de pareamento. No contexto deste trabalho, pares complementares A-T, TA, C-G e G-C recebem +2 pontos; pares não complementares recebem -1; e o alinhamento de uma base com uma lacuna recebe -2.

O objetivo é encontrar uma forma de alinhar as duas sequências de modo que a pontuação final seja a maior possível. Esse tipo de problema é relevante em bioinformática, pois permite comparar estruturas genéticas, identificar similaridades entre organismos e estudar relações evolutivas.

O enunciado solicita a implementação e a avaliação de dois paradigmas de solução: um algoritmo guloso e um algoritmo de programação dinâmica. A comparação entre esses

paradigmas permite observar, na prática, o trade-off entre custo computacional e qualidade da solução obtida.

2. Metodologia

O projeto foi desenvolvido a partir do código-base fornecido pelo professor. Apenas os arquivos `algoritmos.c` e `algoritmos.h` foram modificados, mantendo-se a lógica de leitura de arquivo, medição de tempo e impressão de resultados implementada em `base.c`.

2.1 Função de pontuação

A função `score(a, b)` implementa diretamente as regras do problema. Quando as bases são complementares (A-T, T-A, C-G e G-C), a função retorna 2; quando uma das posições contém uma lacuna '-', a função retorna -2 e, nos demais casos, o retorno é -1. Sendo essa função utilizada por ambos os algoritmos.

A implementação no código é ilustrada por:

```
if ((a == 'A' && b == 'T') || ...)
```

```
    return 2;
```

```
if (a == '-' || b == '-')
```

```
    return -2;
```

```
return -1;
```

Essa função é utilizada por ambos os algoritmos, garantindo consistência na avaliação das soluções.

2.2 Algoritmo Guloso

O algoritmo guloso implementado na função `guloso(char *s1, char *s2)` realiza o alinhamento das sequências tomando decisões baseadas exclusivamente no ganho imediato de cada passo.

2.2.1 Inicialização

Inicialmente, são definidos:

- `i` e `j`: índices para percorrer as sequências
- `n` e `m`: tamanhos das strings (via `strlen`)

- pontuacao: acumulador do resultado
- `int i = 0, j = 0;`
- `long int pontuacao = 0;`
- `int n = strlen(s1);`
- `int m = strlen(s2);`

2.2.2 Estrutura de decisão gulosa

O algoritmo executa um laço: `while (i < n || j < m)`

Isso significa que ele continua até consumir completamente ambas as sequências. Assim, a cada iteração, são avaliadas três possibilidades:

- Match/Mismatch (diagonal)
- Gap em s2 (consome s1)
- Gap em s1 (consome s2)

Para evitar escolhas inválidas, os valores são inicializados com um número muito baixo:

```
int match = -9999;
```

```
int gap_s1 = -9999;
```

```
int gap_s2 = -9999;
```

2.2.3 Cálculo das opções

As opções são calculadas apenas se válidas:

```
if (i < n && j < m)
```

```
    match = score(s1[i], s2[j]);
```

```
if (i < n)
```

```
    gap_s1 = GAP;
```

```
if (j < m)
```

```
    gap_s2 = GAP;
```

2.2.4 Escolha gulosa

A decisão é feita escolhendo a maior pontuação imediata:

```
if (match >= gap_s1 && match >= gap_s2) {
```

```
    pontuacao += match;
```

```
    i++;
```

```

    j++;
}
else if (gap_s1 >= gap_s2) {
    pontuacao += gap_s1;
    i++;
}
else {
    pontuacao += gap_s2;
    j++;
}

```

Essa escolha local favorece o alinhamento direto sempre que possível, já que mismatch (-1) ainda é melhor que gap (-2)

2.2.5 Características do algoritmo e resumo

O algoritmo guloso percorre as duas sequências simultaneamente. Em cada passo, ele escolhe a melhor decisão local entre três possibilidades: alinhar as duas bases atuais, inserir uma lacuna em s1 ou inserir uma lacuna em s2. Consequentemente, como a escolha considera apenas o ganho imediato, o método é rápido, porém pode ignorar o impacto futuro das decisões, caracterizando a chamada miopia algorítmica, na qual escolhas localmente ótimas podem prejudicar o resultado global.

No código implementado, o algoritmo guloso possui complexidade de tempo $O(n + m)$ e consumo de memória $O(1)$, pois utiliza apenas variáveis auxiliares e não armazena soluções intermediárias.

2.3 Programação dinâmica

A solução por programação dinâmica constrói uma matriz dp em que cada posição $dp[i][j]$ representa a melhor pontuação possível para alinhar os prefixos $s1[0..i-1]$ e $s2[0..j-1]$. Cada estado é calculado a partir do máximo entre três opções: diagonal (alinhar as bases atuais), cima (inserir lacuna em s2) e esquerda (inserir lacuna em s1).

2.3.1 Estrutura da Matriz

A matriz é inicializada com penalidades acumuladas de lacunas na primeira linha e na primeira coluna e ao final, a resposta ótima é obtida em `dp[n][m]`. Por fim, na implementação final, a matriz foi alocada dinamicamente com `malloc`, evitando estouro de pilha para instâncias maiores.

```
long int **dp = malloc((n+1) * sizeof(long int*));  
for (int i = 0; i <= n; i++) {  
    dp[i] = malloc((m+1) * sizeof(long int));  
}
```

2.3.2 Inicialização

Casos base: `dp[0][0] = 0;`

- Primeira coluna (gaps em `s2`):

```
for (int i = 1; i <= n; i++)  
    dp[i][0] = dp[i-1][0] + GAP;
```

- Primeira linha (gaps em `s1`):

```
for (int j = 1; j <= m; j++)  
    dp[0][j] = dp[0][j-1] + GAP;
```

2.3.3 Recorrência

Para cada posição (i, j) :

- Diagonal:

```
dp[i-1][j-1] + score(s1[i-1], s2[j-1])
```

- Cima:

```
dp[i-1][j] + GAP
```

- Esquerda:

```
dp[i][j-1] + GAP
```

O valor é escolhido pela função auxiliar: `dp[i][j] = maiorTres(diagonal, cima, esquerda);`

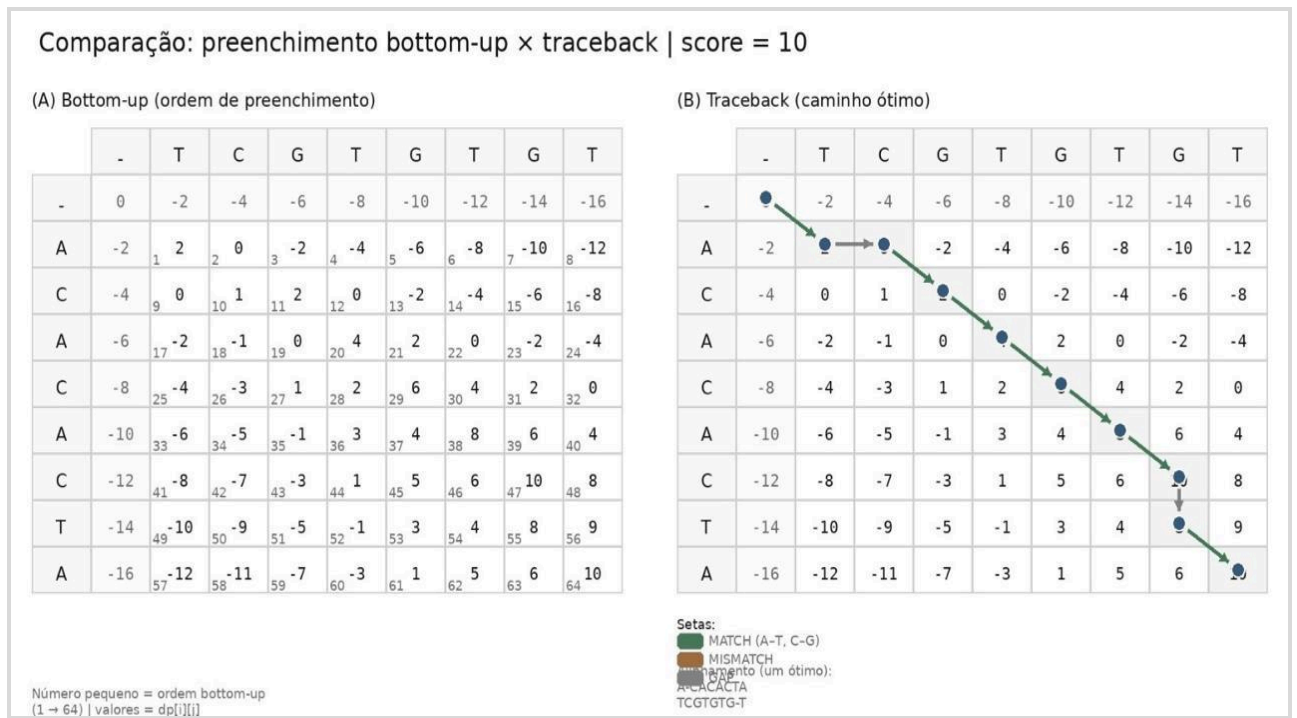


Figura 1. Ilustração do preenchimento da matriz de programação dinâmica (bottom-up) e do processo de traceback para obtenção do alinhamento ótimo entre duas sequências de DNA

2.3.4 Função auxiliar

long int maiorTres(long int a, long int b, long int c)

Essa função retorna o maior entre três valores, centralizando a lógica de decisão.

2.3.5 Resultado final

A solução ótima está na posição: `dp[n][m]`

2.3.6 Liberação de memória

Após o uso, toda memória alocada é liberada:

```
for (int i = 0; i <= n; i++)
```

```
    free(dp[i]);
```

```
free(dp);
```

2.3.7 Propriedades do algoritmo e resumo

Como resultado, o algoritmo obteve complexidade de tempo igual a $O(nm)$ e o consumo de memória também igual a $O(nm)$. Tendo isso em vista, a programação dinâmica garante a solução ótima devido a:

- Subestrutura ótima
- Sobreposição de subproblemas
- Exploração completa do espaço de soluções

2.4 Instâncias e coleta de dados

Foram utilizadas 25 instâncias distribuídas em cinco tamanhos de entrada: 100, 500, 1000, 5000 e 10000 caracteres, com cinco arquivos para cada tamanho. Para cada instância, registrou-se a pontuação final de pareamento e o tempo de execução dos dois algoritmos. Os tempos foram medidos pelo código-base com `gettimeofday`.

2.5 Estimativa de memória

Como o código-base disponibilizado mede automaticamente o tempo, mas não mede o uso de memória, o gráfico de memória deste relatório foi produzido a partir de estimativa teórica baseada na implementação. Para o algoritmo guloso, o consumo é constante, pois apenas algumas variáveis escalares são usadas. Para a programação dinâmica, o consumo foi estimado pela matriz `dp` de `long int` alocada dinamicamente: aproximadamente $8 \times (n+1) \times (m+1)$ bytes, além do vetor de ponteiros para as linhas.

A título de exemplo, para uma entrada de tamanho $n = m = 10000$, a matriz `dp` possui aproximadamente (10001×10001) posições. Considerando que cada posição armazena um valor do tipo `long int` (8 bytes), o consumo aproximado de memória é: $8 \times (10001 \times 10001) \approx 800$ MB

3. Gráficos

Os gráficos a seguir utilizam a média dos cinco testes de cada tamanho de entrada. No gráfico de memória, o valor do guloso é uma estimativa constante e o valor da programação dinâmica é uma estimativa teórica derivada da estrutura de dados usada no código.

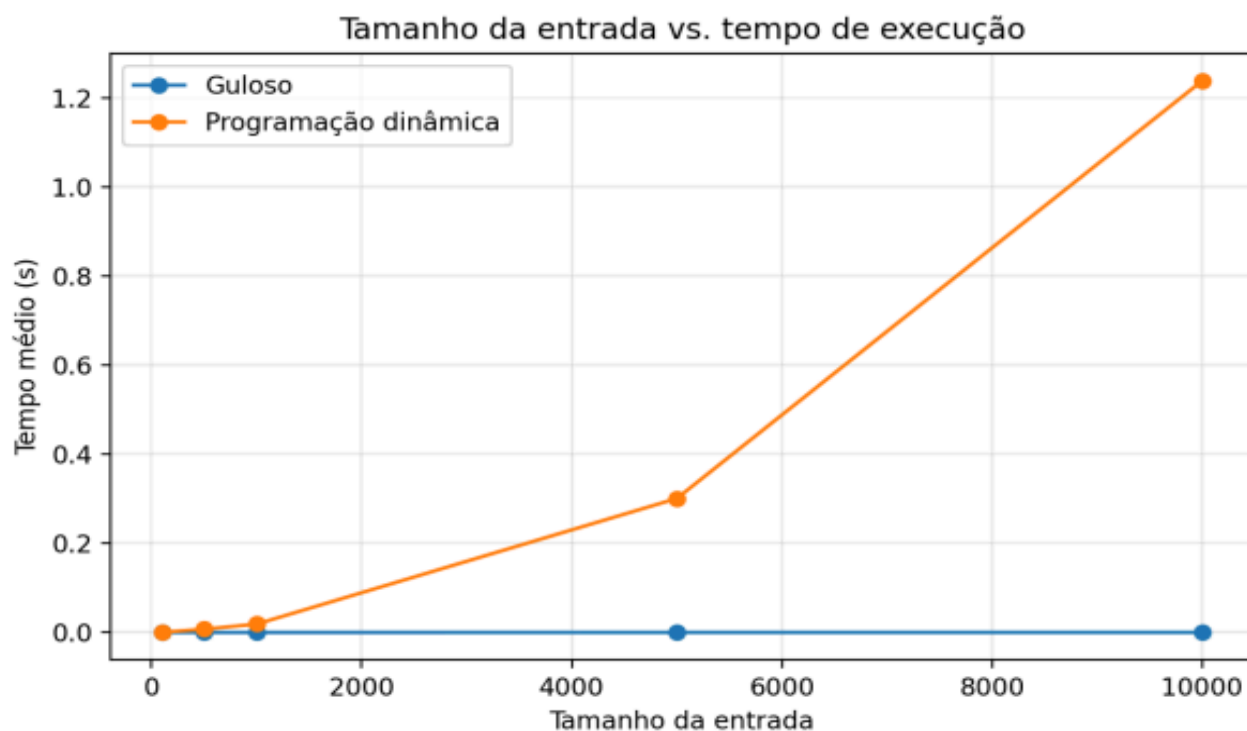


Figura 2. Relação entre o tamanho da entrada e o tempo de execução do algoritmo guloso e de programação dinâmica.

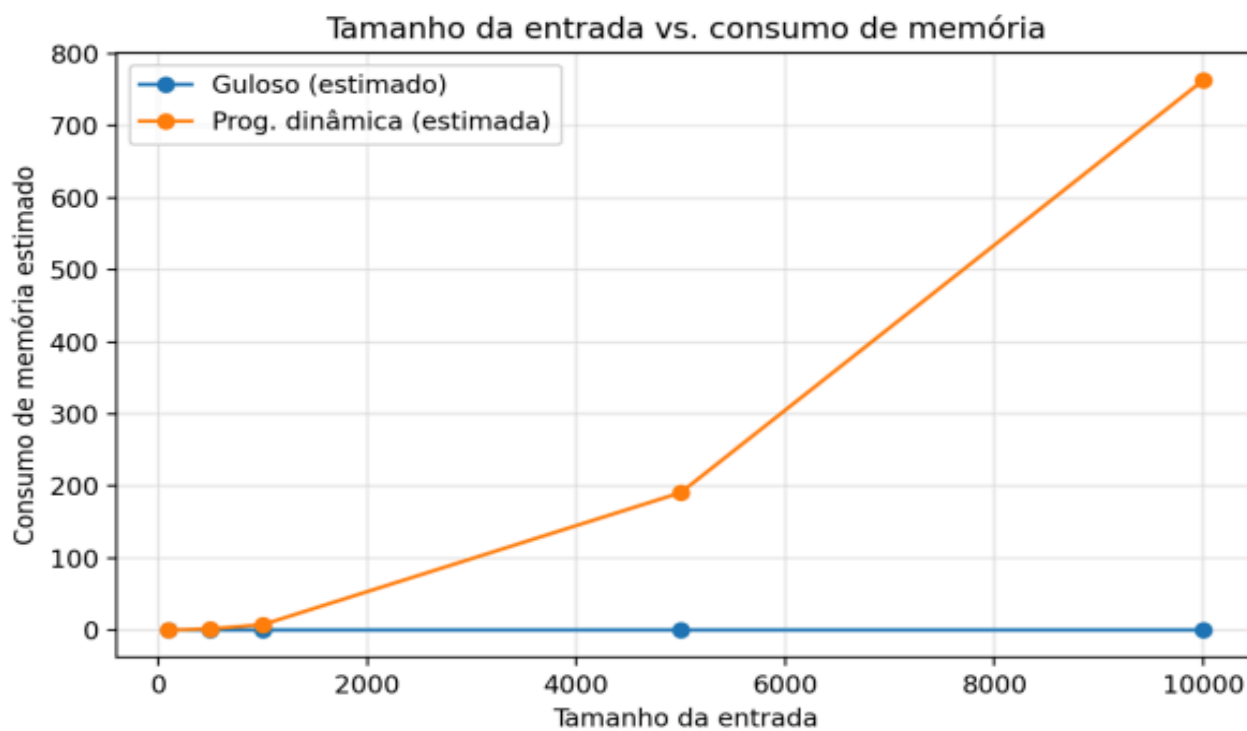


Figura 3. Relação entre o tamanho da entrada e o consumo de memória dos algoritmos guloso e de programação dinâmica

4. Tabela de Pontuação

A tabela abaixo reúne, para cada instância, a pontuação e o tempo medidos para os dois algoritmos. Os resultados mostram que a programação dinâmica obteve pontuação superior em todas as instâncias testadas.

Tabela 1. Resultados dos algoritmos para diferentes instâncias

Instância	Guloso (Pontuação)	Guloso (Tempo s)	Prog. Dinâmica (Pontuação)	Prog. Dinâmica (Tempo s)
100-1	-40	0.000005	32	0.000372
100-2	-13	0.000003	38	0.000378
100-3	-28	0.000005	35	0.000384
100-4	-37	0.000002	32	0.000436
100-5	-4	0.000005	41	0.000444
500-1	-122	0.000009	205	0.008999
500-2	-119	0.000010	196	0.007217
500-3	-149	0.000010	187	0.007156
500-4	-161	0.000009	199	0.007065
500-5	-95	0.000007	196	0.006232
1000-1	-340	0.000014	395	0.018961
1000-2	-274	0.000017	410	0.019697
1000-3	-283	0.000014	446	0.017405
1000-4	-277	0.000017	389	0.018402
1000-5	-235	0.000016	401	0.019144

5000-1	-1271	0.000080	2155	0.301028
5000-2	-1208	0.000086	2149	0.300926
5000-3	-1253	0.000076	2200	0.294101
5000-4	-1247	0.000080	2212	0.309514
5000-5	-1286	0.000092	2155	0.297698
10000-1	-2758	0.000155	4421	1.219503
10000-2	-2572	0.000141	4436	1.234153
10000-3	-2518	0.000138	4451	1.244449
10000-4	-2347	0.000134	4475	1.235890
10000-5	-2434	0.000142	4433	1.254581

4.1 Médias por tamanho

Tabela 2. Médias de pontuação e tempo por tamanho de entrada

Tamanho da Entrada	Média Guloso (Pontuação)	Tempo Médio Guloso (s)	Média Prog. Dinâmica (Pontuação)	Tempo Médio Prog. Dinâmica (s)
100	-24.4	0.000004	35.6	0.000403
500	-129.2	0.000009	196.6	0.007334
1000	-281.8	0.000016	408.2	0.018722
5000	-1253.0	0.000083	2174.2	0.300653

10000

-2525.8

0.000142

4443.2

1.237715

Observação: ressalta-se que os tempos medidos podem variar de acordo com o ambiente de execução, como hardware e carga do sistema. Ademais, o consumo de memória da programação dinâmica foi estimado teoricamente, uma vez que não foi medido diretamente pelo código-base.

5. Análise crítica

Os resultados confirmam o comportamento esperado para os dois paradigmas. O algoritmo guloso foi muito mais rápido em todos os tamanhos testados, mantendo tempos extremamente baixos mesmo para entradas de 10000 caracteres. Essa eficiência decorre do fato de que o método percorre as sequências uma única vez e realiza apenas escolhas locais.

Por outro lado, a rapidez do guloso teve impacto direto na qualidade das respostas. Em todas as instâncias, sua pontuação foi consideravelmente inferior à obtida pela programação dinâmica. Em vários casos, o guloso inclusive produziu pontuações negativas, o que mostra que decisões localmente boas nem sempre conduzem a um alinhamento global satisfatório.

Já a programação dinâmica apresentou o melhor resultado de pontuação em todas as instâncias, pois considera sistematicamente as melhores soluções para subproblemas menores antes de decidir cada passo do alinhamento. Essa estratégia garante otimalidade, mas exige maior custo computacional. Em relação ao crescimento do tempo, nesse caso foi bem mais acentuado à medida que o tamanho da entrada aumentou, especialmente entre 5000 e 10000 caracteres.

Em termos de memória, a diferença entre os algoritmos também é significativa. O guloso usa memória constante, enquanto a programação dinâmica precisa armazenar a matriz completa de subproblemas. Para sequências de 10000 caracteres, a estrutura principal já demanda centenas de megabytes, o que evidencia claramente o trade-off entre desempenho e qualidade da solução.

Assim, conclui-se que o algoritmo guloso é vantajoso quando o principal objetivo é obter uma resposta rápida com baixo custo computacional, aceitando perda de qualidade. Já a

programação dinâmica, por sua vez, é mais adequada quando se busca a melhor pontuação possível e há recursos computacionais suficientes para suportar o maior tempo de execução e o maior uso de memória.

6. Divisão de tarefas

A execução do trabalho foi realizada de forma colaborativa entre os integrantes do grupo, com a seguinte divisão de responsabilidades:

- A elaboração do relatório ficou sob responsabilidade de **Isadora R. Grandaux** e **Otávio de Oliveira**, incluindo a organização do conteúdo, análise dos resultados e redação final do documento.
- O desenvolvimento e implementação dos algoritmos foram conduzidos por **Pedro Brassi Luccas** e **Isabela M. Andrade**, abrangendo tanto o algoritmo guloso quanto o de programação dinâmica.
- A montagem da apresentação em slides, bem como a revisão final do relatório, ficaram a cargo de **Júlia A. da Rocha**, garantindo a clareza, coesão e adequação do material apresentado.

7. Conclusão

O trabalho permitiu comparar, na prática, dois paradigmas clássicos de resolução de problemas: o guloso e a programação dinâmica. Em relação à avaliação experimental, esta mostrou que o algoritmo guloso é muito eficiente em tempo e memória, mas entrega soluções de menor qualidade. Em contrapartida, a programação dinâmica, embora mais custosa, produz alinhamentos significativamente melhores, confirmando sua adequação para cenários em que a otimalidade é essencial. Dessa forma, os experimentos atenderam ao objetivo proposto pelo enunciado e evidenciaram de forma clara o trade-off entre eficiência e qualidade do resultado.

8. Referências

Ching Zhang, Andrew K.C. Wong, A genetic algorithm for multiple molecular sequence alignment, *Bioinformatics*, Volume 13, Issue 6, December 1997, Pages 565–581

CRISTINO, Alexandre dos Santos. Principais algoritmos de alinhamento de sequências genéticas. <<http://www.ime.usp.br/~alexsc>>.

IOSTE, Aline Rodigheri. Sequências de DNA: uma nova abordagem para o alinhamento ótimo. 2016. 148 f. Dissertação (Mestrado em Tecnologias da Inteligência e Design Digital) - Programa de Estudos Pós-Graduados em Tecnologias da Inteligência e Design Digital da Pontifícia Universidade Católica de São Paulo, São Paulo, 2016.